CHAPTER 3

Classes, Objects, and Variables

From the examples we've shown so far, you may be wondering about our earlier assertion that Ruby is an object-oriented language. Well, here is where we justify that claim. We're going to be looking at how you create classes and objects in Ruby and at some of the ways that Ruby is more flexible than other object-oriented languages.

As we said on page 17, everything we manipulate in Ruby is an object. And every object in Ruby was instantiated either directly or indirectly from a class. In this chapter, we'll look in more depth at creating and manipulating those classes.

Defining Classes

Let's give ourselves a simple problem to solve. Suppose we're running a secondhand bookstore. Every week, we do stock control. A gang of clerks uses portable bar-code scanners to record every book on our shelves. Each scanner generates a simple comma-separated value (CSV) file containing one row for each book scanned. The row contains (among other things) the book's ISBN and price. An extract from one of these files looks something like this:

tut_classes/stock_stats/data.csv

```
"Date","ISBN","Price"
"2013-04-12","978-1-9343561-0-4",39.45
"2013-04-13","978-1-9343561-6-6",45.67
"2013-04-14","978-1-9343560-7-4",36.95
```

Our job is to take all the CSV files and work out how many of each title we have, as well as the total list price of the books in stock.

Whenever you're designing an Object-Oriented system, a good first step is to identify the *domain concepts* you're dealing with. Typically the domain concepts—which could represent a physical object, or a process, or some other kind of entity—become classes in your final program, and then individual examples of those concepts are instances of these classes.

It seems pretty clear that we'll need something to represent each data reading captured by the scanners. Each instance of this class will represent a particular row of data, and the collection of all of these objects will represent all the data we've captured.

Let's call this class `BookInStock`. (Remember, class names start with an uppercase letter, and method names normally start with a lowercase letter.) Here's how we declare that class in Ruby: